

- multiple interval selection: you choose several clumps of items that are not continuous. This is the default setting.

The `JList` class delegates the job of showing the list and handling its events. Therefore, the list's model keeps a list of objects, that are rendered in list "cells" by the (VIEW), a "list cell renderer."

A `JList` can display an `Icon` and a `String`—but not both at the same time. The `DefaultListCellRenderer` can only display a single object at one time. If you implement your own custom subclass from the `JList`, you can make it do special things. Right out of the box, the `JList` does not support scrolling, but—just as with the `JTextArea`—you can add it to a `JScrollPane` object. You place the `JList` inside a `JScrollPane`, and the list will scroll. When the user clicks on an item in the `JList`, a `ListSelectionEvent` occurs.

```
Object[] items = { "item1", "item2", "item3", "item4" };
JList list = new JList( items );
list.setVisibleRowCount( 4 );
list.setSelectionMode( ListSelectionModel.SINGLE_
SELECTION );
list.addListSelectionListener( this );
public void valueChanged( ListSelectionEvent e )
{
}
```

Instantiate your `JList`. Set parameters, register the `ListSelectionListener` as your listener and override the inherited abstract method from the `Listener` interface, and use the information contained in the `ListSelectionEvent` object. (Only one method to override)

Any complexity that arises in this object comes from the chance that multiple rows may have been selected.

Constructor:

```
ListSelectionEvent( Object source,
int firstIndex,
int lastIndex,
boolean isAdjusting
);
```

In practice, you would use the same `getSource()` that you have used for the `ActionEvent` object. Both `ActionEvent` and `ListSelectionEvent` inherit this same `getSource()` method from `EventObject`. However, you will need to cast this back into a `JList` object to really take advantage of the information it contains.